HR Automation Dashboard SOP

Architecture

The solution runs entirely on Google Sheets, Apps Script, HtmlService, and MailApp.

```
Source Tabs
  India Employee Database
  US Employee Database
  RM Data
  Finance-Productivity
  Risk Report
  Offboarded Resources
        |
        v
Apps Script Pipeline
  config -> dataLoader -> validation -> alerts/metrics
        |
        +--> Sheet Surfaces: Dashboard, OrgChart, DrillDown
        +--> Web App API: getDashboardData(), getFilteredEmployees(), getLastPipelineStatus()
        +--> Notifications: HR alert digest and admin self-monitoring alerts
        +--> Audit Trail: Logs and Changelog
```

Key modules:

- `config.gs`: tab names, aliases, config defaults, constants, and shared contracts.
- `dataLoader.gs`: header-agnostic source readers, normalization, schema drift warnings, and formula sanitization.
- `alerts.gs`: dashboard model, LWD alerts, probation alerts, attrition, risk summary, and HR Health Score.
- `dashboardRenderer.gs`, `orgChart.gs`, `drillDown.gs`: Google Sheet output surfaces.
- `pipeline.gs`: setup, trigger entrypoints, orchestration, changelog, and partial-failure handling.
- `workflows.gs`: HR workflow actions and idempotency stamps.
- `webApp.gs`, `Index.html`, `Client.html`, `Styles.html`: authenticated HtmlService dashboard.
- `tests.gs`: runnable acceptance harness and QA helpers.

Daily Usage Guide

HR users work in the source tabs only:

- `India Employee Database`
- `US Employee Database`
- `RM Data`
- `Finance-Productivity`
- `Risk Report`

- `Offboarded Resources`

Generated tabs are script-owned:

- `Dashboard`
- `OrgChart`
- `DrillDown`
- `Logs`
- `Changelog`
- `_Config`

Normal workflow:

1. Add or update employee, RM, productivity, risk, or offboarding rows in the relevant source tab.
2. Installable edit/change triggers refresh the generated Sheet surfaces.
3. `Dashboard` shows KPIs, alerts, department breakdown, attrition, risk summary, data quality warnings, and HR Health Score.
4. `OrgChart` reflects reporting-manager relationships.
5. `DrillDown` filter cells can be edited to refresh filtered employee rows.
6. The Web App polls every 5 seconds and reads the same shared dashboard model as the Sheet dashboard.
7. `Logs` records readable operational messages. `Changelog` records one row per full pipeline run.

Threshold Modification Guide

Administrators update thresholds in `_Config`. The pipeline validates values and falls back to defaults when a setting is invalid.

Important keys:

- `LWD_ALERT_DAYS`: days before or after an intern LWD to show an LWD alert.
- `PROBATION_ALERT_DAYS`: days before confirmation date to show a probation alert.
- `PROBATION_DURATION_DAYS`: days after DOJ used to compute confirmation date.
- `PRODUCTIVITY_TARGET`: productivity score below which an employee is flagged.
- `HR_DIGEST_EMAIL`: alert digest recipient.
- `ADMIN_ALERT_EMAIL`: self-monitoring alert recipient.
- `PIPELINE_TRIGGER_HOUR`: daily trigger hour.
- `EMPLOYEE_ID_PATTERN`: validation regex for employee IDs.

After changing `_Config`, run `runFullPipeline("manual")` or edit a source tab row to refresh outputs.

Real-Time Reflection

Sheet reflection:

- Source-tab edits route through `handleInstallableEdit(e)`.
- Source-tab structure changes route through `handleInstallableChange(e)`.
- `DrillDown` filter edits refresh only the filtered output.
- Daily scheduled runs use `runDailyPipeline()`.

Web App reflection:

- `doGet(e)` serves the HtmlService shell.
- `getDashboardData()` loads config, loads source data, builds the shared dashboard model, escapes returned strings, and sends the payload to the browser.
- `getFilteredEmployees(filters)` applies the same drill-down filters used by the Sheet surface.
- The browser polls `getDashboardData()` every 5 seconds.

Partial failures are isolated. For example, MailApp quota errors are logged and surfaced as warnings, but dashboard rendering continues.

Trigger Installation

Run `setupWorkbook()` once before trigger installation to create tabs, seed `_Config`, and protect generated surfaces.

Run `installTriggers()` once as the deployment owner. It installs:

- installable `onEdit` trigger
- installable `onChange` trigger
- daily time-based trigger for `runDailyPipeline()`

Use `removeProjectTriggers()` before reinstalling triggers during maintenance.

Recommended sequence:

1. Open the Apps Script project attached to the workbook.
2. Confirm `appsscript.json` uses V8 and the required scopes.
3. Run `setupWorkbook()`.
4. Run `runAllTests()`.
5. Run `installTriggers()`.
6. Run `runFullPipeline("manual")`.

7. Review `Logs`, `Changelog`, `Dashboard`, `OrgChart`, and `DrillDown`.

Web App Deployment

Deploy from Apps Script as a Web App with `doGet(e)` as the entrypoint.

Deployment settings:

- Execute as: Me
- Access: signed-in organization users
- Manifest webapp setting: `executeAs` = `USER_DEPLOYING`, `access` = `DOMAIN`
- Client API functions: `getDashboardData()`, `getFilteredEmployees(filters)`, and `getLastPipelineStatus()`

Deployment steps:

1. Apps Script > Deploy > New deployment.
2. Select type: Web app.
3. Description: `HR Automation Dashboard`.
4. Execute as: Me.
5. Who has access: signed-in organization users.
6. Deploy and copy the Web App URL.
7. Open the URL as a signed-in organization user.
8. Open the URL in an unauthenticated incognito session and confirm Google redirects to authentication.
9. Record the deployed URL in the submission notes.

No stable deployed Web App URL is stored in source control because Apps Script generates it at deployment time.

Workflow Actions

Workflow action columns are added to employee source tabs when missing.

Supported actions:

- `Start Offboarding`
- `Schedule PIP`
- `Approve Confirmation`

Each action is idempotent. After an action executes, the row receives execution timestamp and result stamps. Re-selecting the same action on the same stamped row does not resend email or repeat side effects.

HR Health Score

The HR Health Score is a 0-100 composite score built from the shared dashboard model. Components are exposed separately in the Sheet Dashboard and Web App.

Weights:

- Attrition: 30%
- Open alerts: 25%
- Active risks: 25%
- Productivity average: 20%

Component scoring:

- Attrition component = `max(0, 100 - attritionRate * 5)`
- Open alerts component = `max(0, 100 - alertCount * 10)`
- Active risks component = `max(0, 100 - activeRiskCount * 15)`
- Productivity component = productivity average

Final score:

```
sum(componentScore * componentWeight / 100)
```

The final result is rounded and clamped between 0 and 100.

Acceptance Verification

Run `runAllTests()` from Apps Script after deploying the latest code. The harness writes one `PASS ...` or `FAIL ...` row per test to `Logs`, followed by a summary and acceptance report line.

Automated checks covered by `runAllTests()`:

- LWD alerts
- Probation alerts
- Current-quarter attrition math
- Column reorder and alias resilience
- Config threshold changes
- Workflow idempotency
- Formula sanitization
- HtmlService escaping

- Web App payload and filter behavior
- Partial MailApp failure behavior

Manual checks to complete in the workbook and deployed Web App:

- Add three QA employees using IDs generated by `createQaTestDataSet_()` and confirm Sheet Dashboard and Web App counts update.
- Change a probation employee DOJ and confirm probation alerts update.
- Delete a QA offboarded row and confirm attrition updates.
- Change `_Config` thresholds and confirm alert/productivity counts update.
- Enter an invalid employment status and confirm a readable validation message appears.
- Open the deployed Web App as a signed-in organization user.
- Open the deployed Web App in an unauthenticated incognito session and confirm Google redirects to authentication.

QA cleanup must only remove rows where `isQaTestEmployeeId_(employeeId)` returns true. Do not delete or overwrite non-test employee IDs.

Security Review

Reviewed controls:

- No credentials, API keys, tokens, passwords, or service account secrets are stored in source code.
- No `UrlFetchApp`, external database, external API, or third-party network integration is used.
- MailApp is the only outbound integration.
- Compensation fields are loaded for internal normalization but are not rendered into dashboard rows, Web App public employee rows, logs, or HR alert digest bodies.
- HtmlService responses recursively escape Sheet-provided strings before returning payloads to the browser.
- Inline initial payload JSON escapes <, >, &, U+2028, and U+2029 to prevent script-tag breakout.
- Client rendering uses `textContent`; it does not use `innerHTML`, `document.write`, `eval`, or dynamic code execution.
- Generated tabs and `_Config` are protected by `setupWorkbook()`.
- Web App access is restricted to signed-in organization users.
- Broad iframe embedding is not enabled in code.

Reviewer access:

- Grant workbook reviewers view-only access unless they are expected to run admin setup.
- Grant Apps Script edit access only to maintainers.
- Deployment owner should be a controlled Workspace account, not a personal account.

Troubleshooting

Check `Logs` first. Messages are plain English and include INFO, WARN, or ERROR levels.

Common issues:

- Missing source tab: run `setupWorkbook()`.
- Missing or renamed header: restore the canonical header or add an alias in `config.gs`.
- Invalid config value: correct the `_Config` row; the pipeline falls back to defaults until fixed.
- Mail quota exceeded: dashboard rendering continues; digest failure is logged and admin self-monitoring is attempted.
- Web App shows an error view: open `Logs`, fix the reported workbook/config/source issue, and refresh.
- Trigger did not run: run `removeProjectTriggers()`, then `installTriggers()` as the deployment owner.

Change Management

Use one git checkpoint per completed pass. Do not push mid-pass.

Recommended release process:

1. Run `runAllTests()`.
2. Run static parser checks locally when editing source files.
3. Deploy Apps Script.
4. Run the manual acceptance checks.
5. Commit with the pass checkpoint message.
6. Push to `origin/main`.
7. Record final Web App URL and workbook URL in submission notes outside source control.

Submission Checklist

- Source code pushed to GitHub.
- Apps Script project opened without syntax errors.
- `setupWorkbook()` completed.
- `installTriggers()` completed.
- `runAllTests()` passed.
- Manual acceptance checks completed.
- Web App deployed with the settings above.

- Reviewer has view-only access to workbook and Web App access through organization sign-in.
- `SOP.pdf` generated from this SOP.